Git & GitHub — Hands On

1. What Are Git and GitHub?

Git

Git is a distributed version-control system. It keeps track of changes to your source code and allows you to:

- Save versions of your project.
- See what changed.
- Return to previous versions.
- Create branches for different features.
- Merge changes.
- Work safely without losing previous work.

GitHub

GitHub is an online platform for hosting Git repositories.

Git runs primarily on your computer, while GitHub provides a remote copy of your repository and collaboration features such as:

- Pull Requests
- Code review
- Issues
- Repository hosting
- Team collaboration

A useful way to remember it:

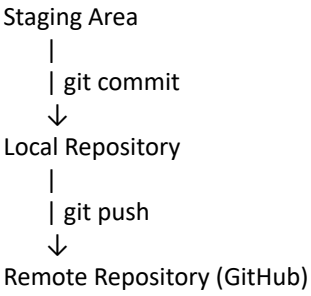
```
Git     = Version control
GitHub  = Online Git hosting + collaboration
```

---

2. Basic Git Concepts

A Git repository has several important concepts:

```
Working Directory
      |
      | git add
      ↓
```



Working Directory

The actual files you are editing.

Staging Area

The changes you've selected for your next commit.

Commit

A snapshot of your project at a particular point in time.

Branch

An independent line of development.

Remote

A Git repository stored somewhere else, commonly GitHub.

---

3. Installing Git

Install Git from the official Git website:

https://git-scm.com/

Verify the installation:

```
git --version
```

Example:

```
git version 2.x.x
```

---

4. Configure Git

Set your name:

```
git config --global user.name "Your Name"
```

Set your email:

```
git config --global user.email "you@example.com"
```

Check the configuration:

```
git config --global --list
```

---

5. Create a Git Repository

Create a project:

```
mkdir my-project
cd my-project
```

Initialize Git:

```
git init
```

Git creates a repository for the project.

Check the repository:

```
git status
```

---

6. Create Your First Commit

Suppose you create:

```
file1.txt
```

Check its status:

```
git status
```

Stage it:

```
git add file1.txt
```

Or stage everything:

```
git add .
```

Create a commit:

```
git commit -m "Initial commit"
```

Your first commit is now stored in Git history.

---

7. Understanding Commits

Every commit has a unique identifier called a **commit hash**.

For example:

```
42531ac Initial commit
5644a8e Create file2.txt
8b6af8c login feature
```

The first part is the commit hash.

You can view your history with:

```
git log --oneline
```

A more detailed view:

```
git log
```

A graphical view:

```
git log --oneline --graph --all --decorate
```

---

8. Git Status

One of the most important commands is:

```
git status
```

It tells you:

- Which branch you're on.
- Which files have changed.
- Which files are staged.
- Which files are untracked.
- Whether your branch is ahead or behind the remote.

Use it frequently.

---

9. Git Diff

git diff shows the changes you have made.

```
git diff
```

This shows changes that are **not yet staged**.

For example:

```
- Hello World
+ Hello Git
```

- means the old content.

+ means the new content.

9.1 View Staged Changes

After:

```
git add .
```

regular git diff may no longer show those changes.

Use:

```
git diff --staged
```

or:

```
git diff --cached
```

These show changes that are staged and ready to commit.

9.2 Compare Everything With the Last Commit

Use:

```
git diff HEAD
```

This compares your current working state with the latest commit.

9.3 Compare Two Commits

For example:

```
git diff 42531ac 5644a8e
```

This shows what changed between those two commits.

9.4 Compare Branches

```
git diff main new-branch
```

This shows the differences between the two branches.

9.5 Show Only a Summary

```
git diff --stat
```

Example:

```
file1.txt | 10 +++++-----
file2.txt |  5 +++++
2 files changed
```

---

10. The Normal Git Workflow

A common workflow is:

```
git status

# Make changes

git diff

git add .

git diff --staged

git commit -m "Describe changes"

git push
```

Before starting work on an existing project, commonly:

```
git pull
```

So the general cycle is:

git pull
  ↓
Make changes
  ↓
git diff
  ↓
git add
  ↓
git diff --staged
  ↓
git commit
  ↓
git push

---

11. Connecting a Local Repository to GitHub

Create a repository on GitHub.

Then connect your local repository:

git remote add origin https://github.com/USERNAME/PROJECT.git

Check it:

git remote -v

You should see something similar to:

origin https://github.com/USERNAME/PROJECT.git (fetch)
origin https://github.com/USERNAME/PROJECT.git (push)

---

12. Push Your Main Branch

Rename your current branch to main if necessary:

git branch -M main

Push it:

git push -u origin main

Notice:

-u

is commonly used with **git push**, not git pull.

The -u sets the upstream branch.

After that, you can normally use:

```
git push
```

and:

```
git pull
```

---

13. Clone a GitHub Repository

If a repository already exists on GitHub:

```
git clone https://github.com/USERNAME/PROJECT.git
```

Then:

```
cd PROJECT
```

You now have a local copy of the GitHub repository.

---

14. Git Pull

git pull gets changes from the remote repository and integrates them into your current branch.

```
git pull
```

If the upstream branch hasn't been configured, Git may say:

```
There is no tracking information for the current branch.
```

You can specify it directly:

```
git pull origin main
```

Or configure the upstream:

```
git branch --set-upstream-to=origin/main main
```

After that:

```
git pull
```

works without specifying the remote and branch.

---

15. Git Push

git push sends your local commits to GitHub.

```
git push
```

For a new branch:

```
git push -u origin new-branch
```

After the upstream is configured:

```
git push
```

is sufficient.

---

16. Branches

Branches allow you to develop features separately from main.

See branches:

```
git branch
```

Create a branch:

```
git branch new-branch
```

Switch to it:

```
git switch new-branch
```

Or create and switch in one command:

```
git switch -c new-branch
```

---

17. Create a Branch From an Earlier Commit

You can create a branch starting at any commit.

For example:

```
8b6af8c  login feature
5644a8e  Create file2.txt
```

```
42531ac  Initial commit
```

Create a branch starting from 5644a8e:

```
git branch new-branch 5644a8e
```

Then switch to it:

```
git switch new-branch
```

These two commands are equivalent to:

```
git switch -c new-branch 5644a8e
```

The resulting history is:

```
8b6af8c login feature      ← main
    |
5644a8e Create file2.txt   ← new-branch
    |
42531ac Initial commit
```

The new branch starts exactly where you specify.

---

18. Branches Do Not Require Pull Requests

Git itself does **not** require Pull Requests.

You can create a branch, work on it, merge it locally, and push the result.

For example:

```
git switch main
git pull

git switch -c new-branch

# Make changes

git add .
git commit -m "Add feature"

git switch main
git merge new-branch

git push
```

No Pull Request is required because you performed the merge locally.

---

19. Pull Request Workflow

A Pull Request is primarily a **GitHub collaboration and code-review mechanism**.

Instead of merging locally:

```
git switch main
git merge new-branch
```

you can push the feature branch:

```
git push -u origin new-branch
```

GitHub can then be used to create a Pull Request:

```
new-branch
    |
    ↓
Pull Request
    |
    ↓
Code Review
    |
    ↓
Merge
    |
    ↓
main
```

A Pull Request allows developers to:

- Review code.
- Comment on changes.
- Request modifications.
- Run automated tests.
- Approve changes.
- Merge the branch.

A repository can also have branch protection rules requiring Pull Requests before changes can enter main.

---

20. Merge a Branch Locally

Suppose you're on main:

```
git switch main
```

Update it:

```
git pull
```

Merge your branch:

```
git merge new-branch
```

Then push:

```
git push
```

The merge happens locally. GitHub receives the resulting history when you push.

---

21. Reverting to an Earlier Version

There are two different concepts:

git reset

Moves a branch pointer backward.

git revert

Creates a new commit that reverses previous changes.

If you want to **keep the history**, use:

```
git revert
```

---

22. Git Revert

Suppose your history is:

A → B → C

and you want to undo C.

Run:

```
git revert C
```

Git creates a new commit:

```
A → B → C → D
        ↑
    Reverts C
```

The original commit C remains in history.

This is useful when you want a rollback without deleting history.

---

23. Revert Back to the Initial Commit While Keeping History

Suppose your history is:

```
8b6af8c  login feature
5644a8e  Create file2.txt
42531ac  Initial commit
```

If you want the project files to return to the state of 42531ac while keeping all commits in history, you can revert the changes after that commit:

```
git revert 42531ac..HEAD
```

Git creates new revert commits.

The history remains:

```
XXXXXXX Revert "login feature"
XXXXXXX Revert "Create file2.txt"
8b6af8c login feature
5644a8e Create file2.txt
42531ac Initial commit
```

The working files will represent the earlier state, but the original commits remain in the history.

---

24. Git Reset

git reset is different from git revert.

For example:

```
git reset --hard 42531ac
```

moves the current branch directly back to 42531ac.

Before:

A → B → C

After:

A

The branch no longer points to B and C.

--hard also discards uncommitted changes to tracked files.

Therefore, be careful with:

```
git reset --hard
```

---

25. Reset vs Revert

Situation	Recommended
Keep history	git revert
Undo a pushed commit	git revert
Local commits haven't been shared	git reset can be appropriate
Move branch pointer backward	git reset
Permanently remove local commit history	git reset
Safely undo changes in a shared branch	git revert

A useful rule:

```
Shared history?
  ↓
Use revert

Private/local history?
  ↓
Reset may be appropriate
```

---

26. Unrelated Histories

Sometimes you may see:

```
fatal: refusing to merge unrelated histories
```

This happens when your local repository and the remote repository were created independently.

For example:

```
Local:
A → B

GitHub:
X → Y
```

Git doesn't consider these repositories to share a common history.

One option is:

```
git pull origin main --allow-unrelated-histories
```

This tells Git that you intentionally want to merge the two separate histories.

However, if the GitHub repository is just a newly created repository containing nothing important, another approach may be to make your local project the remote version.

For example:

```
git push -u origin main --force-with-lease
```

Use this only when you are certain that replacing the remote history is appropriate.

---

27. Example: Two Different Initial Commits

Suppose:

```
e2538e8 (origin/main) Initial commit
42531ac (HEAD -> main) Initial commit
```

Although both commits are named Initial commit, they have different hashes.

That means they were created separately.

Git sees:

```
Local repository:
42531ac

GitHub:
e2538e8
```

rather than:

```
42531ac
  ↓
e2538e8
```

Therefore Git may report:

```
refusing to merge unrelated histories
```

---

28. A Practical Branch Example

Suppose your history is:

```
8b6af8c login feature
5644a8e Create file2.txt
42531ac Initial commit
```

You want a new branch based on Create file2.txt.

You can do:

```
git branch new-branch 5644a8e
git switch new-branch
```

Or simply:

```
git switch -c new-branch 5644a8e
```

Now:

```
          8b6af8c login feature ← main
         /
5644a8e Create file2.txt ← new-branch
         \
42531ac Initial commit
```

You can develop independently from that point.

---

30. Useful Git Commands

Command	Purpose
git init	Create a repository
git status	Show repository status
git add .	Stage changes
git commit -m "message"	Create a commit
git log	Show history
git log --oneline	Compact history
git log --graph --all --decorate	Visualize history
git diff	Show unstaged changes
git diff --staged	Show staged changes
git diff HEAD	Compare current state with HEAD
git branch	List branches
git branch name	Create a branch

Command	Purpose
git switch name	Switch branches
git switch -c name	Create and switch
git merge name	Merge a branch
git revert commit	Undo a commit while keeping history
git reset --hard commit	Move branch backward
git clone URL	Clone a repository
git pull	Get remote changes
git push	Send commits to remote
git remote -v	Show remotes

---

31. A Good Everyday Workflow

For a personal project:

```
git switch main
git pull

git switch -c feature-name

# Make changes

git status
git diff

git add .

git diff --staged

git commit -m "Add feature"

git switch main
git merge feature-name

git push
```

For a team project using Pull Requests:

```
git switch main
git pull

git switch -c feature-name

# Make changes

git add .
git commit -m "Add feature"
```

```
git push -u origin feature-name
```

Then create a Pull Request on GitHub.

After the Pull Request is merged:

```
git switch main
git pull
```

---

32. The Most Important Commands to Memorize

Start with these:

```
git status
git add .
git commit -m "message"
git pull
git push
```

Then learn:

```
git branch
git switch
git merge
git diff
git log
git revert
```

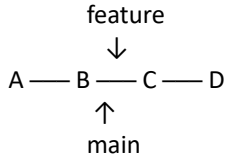
And eventually become comfortable with:

```
git reset
git rebase
git stash
git cherry-pick
```

---

33. Mental Model

The easiest way to understand Git is to think of commits as a chain of snapshots:



A branch is simply a **pointer to a commit**.

When you create a branch:

```
git branch feature B
```

you are essentially saying:

```
feature → B
```

When you make a new commit:

```
A —— B —— C
     ↑    ↑
  feature main
```

The branch pointer moves forward.

This mental model makes commands such as branch, switch, merge, revert, and reset much easier to understand.

---

34. Final Cheat Sheet

Create repository

```
git init
```

Check status

```
git status
```

Save changes

```
git add .
git commit -m "message"
```

View history

```
git log --oneline
```

View changes

```
git diff
```

View staged changes

```
git diff --staged
```

Create branch

git branch new-branch

Create branch from an earlier commit

git branch new-branch 5644a8e

Switch branch

git switch new-branch

Create and switch

git switch -c new-branch

Create branch from an earlier commit and switch

git switch -c new-branch 5644a8e

Merge branch

git switch main
git merge new-branch

Undo a commit while keeping history

git revert <commit>

Move branch backward

git reset --hard <commit>

Connect GitHub

git remote add origin <repository-url>

Push main

git push -u origin main

Push a new branch

git push -u origin new-branch

Pull

git pull

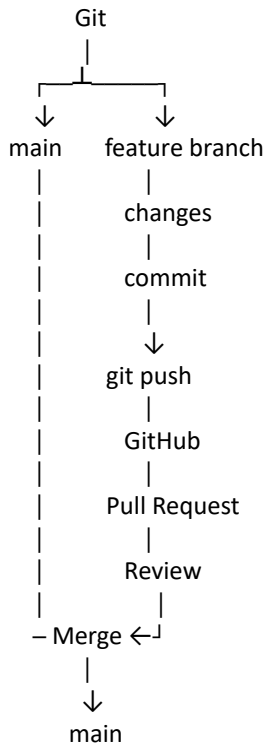
Clone

git clone <repository-url>

---

35. The Big Picture

The complete Git/GitHub workflow can be visualized as:



The key ideas are:

1. **Git tracks versions of your project.**
2. **A commit is a snapshot of your project.**
3. **A branch is a pointer to a line of development.**

4. **git diff lets you inspect changes.**
5. **git revert undoes changes while preserving history.**
6. **git reset moves a branch pointer backward and can rewrite local history.**
7. **Branches can start from any commit.**
8. **Git does not require Pull Requests.**
9. **Pull Requests are a GitHub collaboration/review workflow.**
10. **git push sends local commits to GitHub, while git pull brings remote changes into your local repository.**
11. **Before committing, use git diff and git diff --staged to review what you're about to save.**